Selection-Sort

Selection Sort

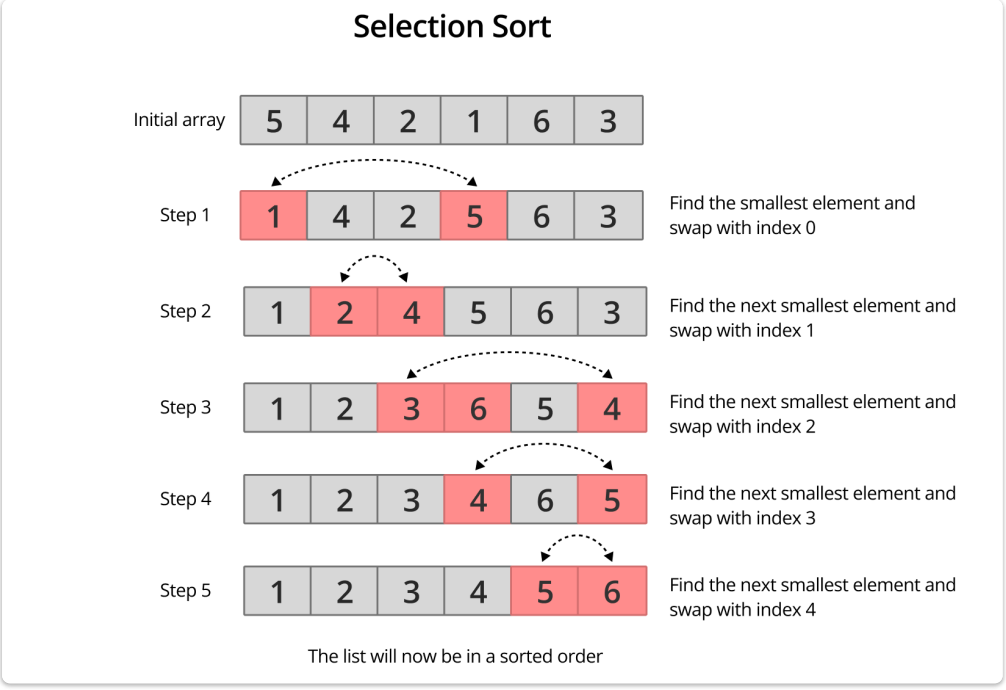
Selection Sort es un algoritmo de ordenación que recorre el array varias veces.
En cada pasada:

1. Busca el elemento más pequeño del tramo que todavía no está ordenado.
2. Lo intercambia con el elemento que está al principio de ese tramo.

Tras la i-ésima pasada, las posiciones `0, 1, 2, ..., i` quedan ya ordenadas, y el segmento desordenado se reduce.

Propiedades clave:

- Tiempo:
  - Mejor, medio y peor caso: (O(n^2))
- Espacio:
  - In-place (usa memoria extra constante).
- Estabilidad:
  - No estable (puede cambiar el orden relativo de elementos iguales).



Esquema Selection Sort

Supongamos un array `A` de longitud `n`.

Idea general del bucle externo:

- Vamos a fijar una posición `i` (desde `0` hasta `n-2`).
- En cada `i`, buscamos el **mínimo** en el tramo `A[i, ..., n-1]`.
- Luego ponemos ese mínimo en la posición `i` intercambiándolo con `A[i]`.

Paso a paso para un `i` concreto:

1. **Inicializar índice del mínimo**
   - `min = i`
   - Interpretación: asumimos que, de momento, el elemento más pequeño del tramo `i..n-1` es el que está en `i`.
2. **Recorrer el resto del tramo no ordenado**
   - Bucle: `j` va de `i+1` a `n-1`.
   - En cada posición `j` comparamos `A[j]` con `A[min]`:
     - Si `A[j] < A[min]`, actualizamos `min = j`.
   - Al terminar el bucle interno, `min` apunta al índice del elemento más pequeño del tramo `A[i..n-1]`.
3. **Intercambio**
   - Intercambiamos `A[i]` con `A[min]`.
   - Resultado: `A[i]` pasa a contener el valor mínimo del tramo.
   - La parte `A[0, ..., i]` queda ordenada (y son los elementos más pequeños del array).
4. **Avanzar el tramo ordenado**
   - Aumentamos `i` y repetimos el proceso con el tramo `A[i+1, ..., n-1]`.

¿Por qué el bucle externo llega solo hasta `n-2`?

- En la penúltima iteración (`i = n-2`) comparamos posiciones `n-2` y `n-1` y dejamos el menor en `n-2`.

- El último elemento (`n-1`) quedará automáticamente colocado, por lo que no hace falta más iteraciones.

Pseudocódigo Selection Sort

```
SELECTION-SORT(A)
    n = length(A)
    for i = 0 to n - 2
        min = i
        for j = i + 1 to n - 1
            if A[j] < A[min]
                min = j
        intercambiar A[i] ↔ A[min]
```

Implementación en Python

```python
def selection_sort(arr):
    n = len(arr)

    # Recorremos todas las posiciones menos la última
    for i in range(n - 1):
        # Suponemos que el mínimo está en la posición i
        min_index = i

        # Buscamos el índice del mínimo en el segmento no ordenado
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Intercambiamos el elemento en i con el mínimo encontrado
        arr[i], arr[min_index] = arr[min_index], arr[i]


# Ejemplo de uso
if __name__ == "__main__":
    unsorted_array = [3, -7, 10, -4, 0, 6, -1, 8, -9, 2]
```

```
    selection_sort(unsorted_array)
    print(unsorted_array)  # [-9, -7, -4, -1, 0, 2, 3, 6, 8, 10]
```